Coding Club

Object Oriented Programming (OOP)

Shihao Shenzhang

Department of Biostatistics & Health Informatics
King's College London

Agenda

- Intro
  - What is a Programming Paradigm?
  - What is Object Oriented Programming (OOP)?
- Class
  - Introduction
  - Object Definition
  - Instance
  - Code Translation
- Inheritance
  - Introduction
  - Subclass & Superclass
  - Code Translation
- Encapsulation
  - Introduction
  - Getters & Setters
  - Benefits

Intro: What is a Programming Paradigm?

Definition

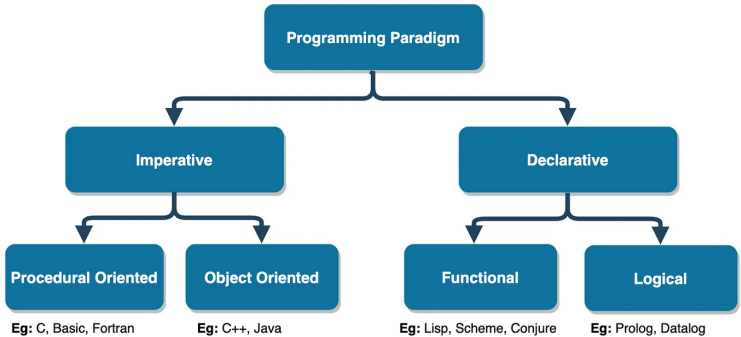
Programming paradigms are fundamental approaches to programming that dictate how developers conceptualise and structure their code.

- Imperative
  - **Describes how to do something**.
  - Examples: C, Java, Python, JavaScript
- Declarative
  - **Describes what you want done**, not how to do it.
  - Examples: Prolog, Conjure, SQL

Intro: What is Object Oriented Programming (OOP)?

Definition

Object-oriented programming (OOP) is a programming paradigm that models a system as a collection of objects, each representing a particular aspect of the system.



Source: `https://pradeesh-kumar.medium.com/programming-paradigms-66248c83b39a`

Example

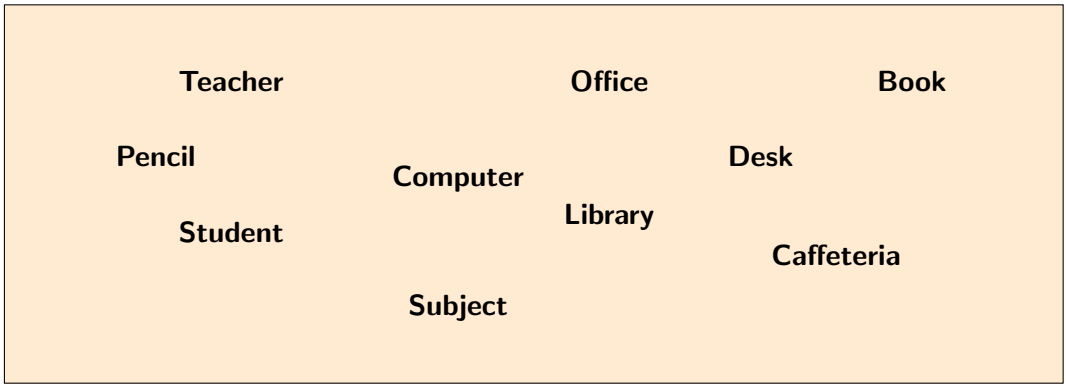
We want to model a school, so our system is the school. What can we have in a school?

System (School)

Example

We want to model a school, so our system is the school. What can we have in a school?

System (School)



Agenda

- Intro
  - What is a Programming Paradigm?
  - What is Object Oriented Programming (OOP)?
- Class
  - Introduction
  - Object Definition
  - Instance
  - Code Translation
- Inheritance
  - Introduction
  - Subclass & Superclass
  - Code Translation
- Encapsulation
  - Introduction
  - Getters & Setters
  - Benefits

Class: Introduction

Definition

In object-oriented programming, a class is a **blueprint** for creating objects, providing initial values for state (attributes), and implementations of behaviour (methods).

In a nutshell, a class is a template for creating objects.

Key components of a class:

- It has a **constructor**, which initialises the object initial state.
- It has **attributes** (characteristics)
- It contains **methods** (actions)

Example:
Teacher (Object)
- **Characteristics**
- **Actions**

Class: Object Definition

Example:
Teacher (Object)

Characteristics (attributes)

- Name
- Gender
- Age
- Seniority
- Subject

Actions (methods)

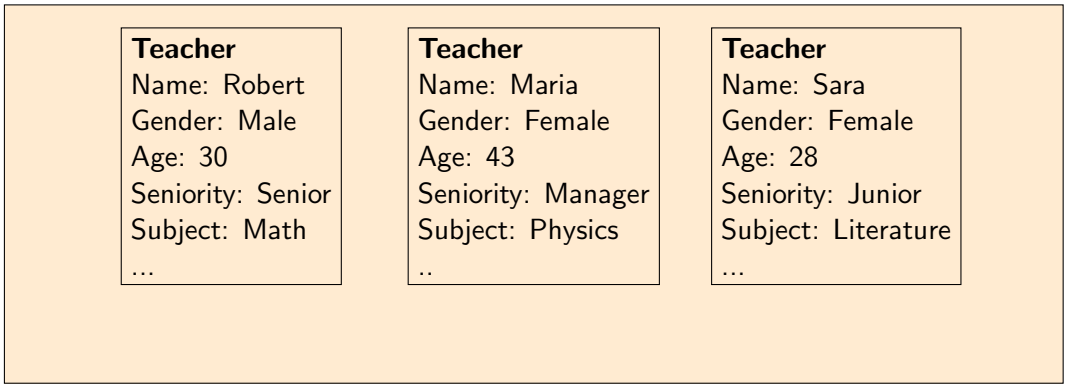
- Introduce
- Teach
- Promote

Teacher

attributes
Name: Robert
Gender: Male
Age: 30
Seniority: Senior
Subject: Math
methods
Introduce
Teach
Promote

Class: Instance

System (School)



Every time we create a new object of a specific class, we call that an instance.

Class: Code Translation

Example:

Teacher (Object)

Characteristics

- Name
- Gender
- Age
- Seniority
- Subject

Actions

- Introduce
- Teach
- Promote

```python
class Teacher:
    def __init__(self, name, gender, age, subject, seniority):
        # attributes of Teacher
        self.name = name
        self.gender = gender
        self.age = age
        self.seniority = seniority
        self.subject = subject

    # methods of Teacher
    def introduce(self):
        print(f"My name is {self.name} and I am {self.age}")

    def teach(self):
        print(f"I am going to teach you {self.subject}")

    def promote(self, new_position: str):
        self.seniority = new_position
        print(f"My new position is {self.seniority}")


```

```python
if __name__ == "__main__":

    john = Teacher("John", "Male", 34, "Maths", "Junior")

    john.introduce() # prints the message "My name is John and I am 34"

```

Agenda

- Intro
  - What is a Programming Paradigm?
  - What is Object Oriented Programming (OOP)?
- Class
  - Introduction
  - Object Definition
  - Instance
  - Code Translation
- Inheritance
  - Introduction
  - Subclass & Superclass
  - Code Translation
- Encapsulation
  - Introduction
  - Getters & Setters
  - Benefits

Inheritance: Introduction

Definition

Inheritance is basically the idea that different classes can have similar components, and in order to avoid repeating code, inheritance is used to link parent classes to descendant classes.

Example:

Class Teacher

- Name
- Gender
- Age
- Seniority
- Subject

- Introduce
- Teach
- Promote

Class Student

- Name
- Gender
- Age
- Seniority
- Subject

- Introduce
- Study
- Promote

Inheritance: Subclass & Superclass

Example:

Class Teacher

- ~~Name~~
- ~~Gender~~
- ~~Age~~
- ~~Seniority~~
- ~~Subject~~

- ~~Introduce~~
- Teach
- ~~Promote~~

Class Student

- ~~Name~~
- ~~Gender~~
- ~~Age~~
- ~~Seniority~~
- ~~Subject~~

- ~~Introduce~~
- Study
- ~~Promote~~

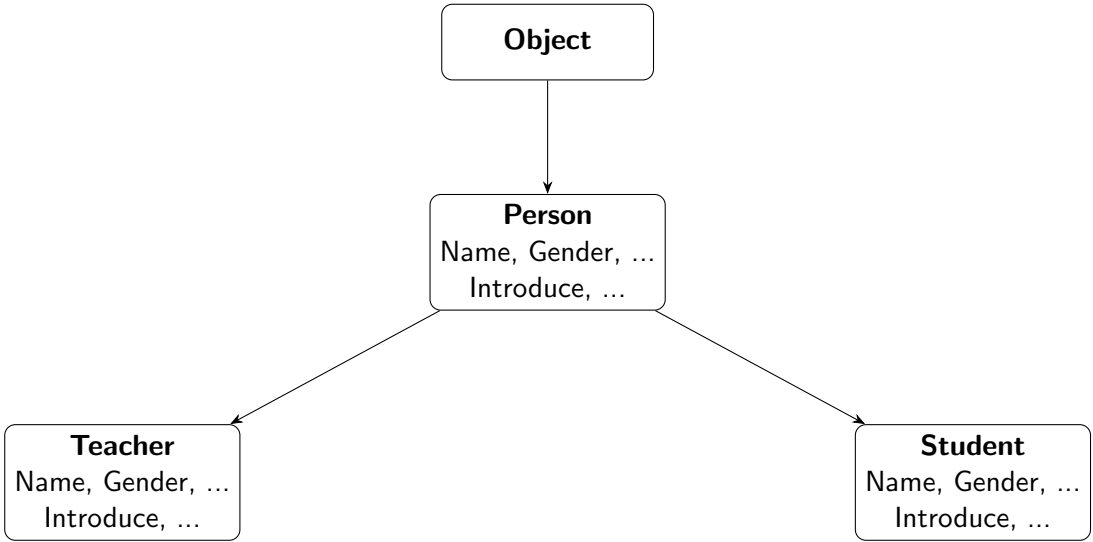
Class Person

- Name
- Gender
- Age
- Seniority
- Subject

- Introduce
- Promote

1) Class Teacher and Student are called **subclasses** because they inherit from Person, and class Person is a **superclass** of Teacher and Student.
2) The subclasses inherit all the methods and attributes from the superclass.

Inheritance: Example



Inheritance: Code Translation I

Class Person

- Name
- Gender
- Age
- Seniority
- Subject

- Introduce
- Promote

```python
class Person:
    def __init__(self, name, gender, age, seniority, subject):
        # attributes of person
        self.name = name
        self.gender = gender
        self.age = age
        self.seniority = seniority
        self.subject = subject

    # methods of person
    def introduce(self):
        print(f"My name is {self.name} and I am {self.age}")

    def promote(self, new_position: str):
        self.seniority = new_position
        print(f"My new position is {self.seniority}")

```

Inheritance: Code Translation II

Class Teacher

- Teach

```python
from person import Person

class Teacher(Person):
    def __init__(self, name, gender, age, seniority, subject):
        # calling the superclass constructor
        super().__init__(name, gender, age, seniority, subject)

    # methods of Teacher
    def teach(self):
        print(f"I am going to teach you {self.subject}")


```

Class Student

- chapter
- Study

```python
from person import Person

class Student(Person):
    def __init__(self, name, gender, age, seniority, subject, chapter):
        # calling the superclass constructor
        super().__init__(name, gender, age, seniority, subject)
        # specific attributes of Student
        self.chapter = chapter

    # methods of Student
    def study(self):
        print(f"I am studying chapter {self.chapter} of {self.subject}")


```

Inheritance: Code Translation III

```
1  if __name__ == "__main__":
2
3      sarah = Student("Sarah", "Female", 18, "1st year", "Maths", 1)
4
5      sarah.introduce() # method inherit from the parent (Person)
6      sarah.study()
7
8
```

Agenda

- Intro
  - What is a Programming Paradigm?
  - What is Object Oriented Programming (OOP)?
- Class
  - Introduction
  - Object Definition
  - Instance
  - Code Translation
- Inheritance
  - Introduction
  - Subclass & Superclass
  - Code Translation
- Encapsulation
  - Introduction
  - Getters & Setters
  - Benefits

Encapsulation: Introduction

Definition

Encapsulation is the practice of bundling an objects data (attributes) and the methods (functions) that operate on that data into a single unit (a class), while restricting direct access to some of the object's components to protect its internal state.

Types of visibility:

- **Public**: visible to everybody.
- **Protected**: visible to the class and its subclasses.
- **Private**: only visible to the class.

Python **soft encapsulates**, meaning that it does not enforce access restrictions strictly (private, protected or public), instead it uses naming conventions.

- public $\rightarrow$ *self.name*
- Protected $\rightarrow$ *self._name*
- Private $\rightarrow$ *self.__name*

Encapsulation: Getters & Setters (VSCode)

What if you want to retrieve an attribute, but you do not want it to be modified?

Getters and setters are methods used in object-oriented programming to control access to an object's attributes.

```python
class Person:
    def __init__(self, name, gender, age, seniority, subject):
        # private attributes of person
        self.__name = name
        self.__gender = gender
        self.__age = age
        self.__seniority = seniority
        self.__subject = subject

    # methods of person
    def introduce(self):
        print(f"My name is {self.name} and I am {self.age}")

    def promote(self, new_position: str):
        self.seniority = new_position
        print(f"My new position is {self.seniority}")

    # getter
    def get_name(self):
        return self.__name

    # setter
    def set_subject(self, new_subject):
        self.__subject = new_subject

```

Encapsulation: Benefits

- **Enhanced security**: Hiding the internal state of objects prevents unauthorised access and manipulation.
- **Increased adaptability**: easier for designers to make changes to the code without risking compatibility.
- **Simplified maintenance**: You can independently develop, test, and debug encapsulated objects.

References

- MDN Web Docs. *Object-oriented programming*. Available at: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Advanced_JavaScript_objects/Object-oriented_programming
- DataCamp. *Introduction to Programming Paradigms*. Available at: https://www.datacamp.com/blog/introduction-to-programming-paradigms
- W3Schools. *Python Classes and Objects*. Available at: https://www.w3schools.com/python/python_classes.asp
- Codecademy. *What is Inheritance?*. Available at: https://www.codecademy.com/resources/blog/what-is-inheritance/
- Brilliant.org. *Classes OOP*. Available at: https://brilliant.org/wiki/classes-oop/
- Coursera. *Encapsulation in Object-Oriented Programming*. Available at: https://www.coursera.org/articles/encapsulation

The End